

Entrega 2 - Introducción a la Ciencia de Datos

Grupo 8: Facundo Morini, Cecilia Waksman

2026-06-18

Introducción

En este trabajo se busca construir un modelo de clasificación de texto capaz de predecir a qué medio de prensa pertenece un artículo periodístico a partir únicamente de su contenido textual. Para esto se utiliza un subconjunto del dataset *All the News 2.1 – Component One*, que reúne artículos de distintos medios junto con los metadatos asociados a cada publicación. El interés no está tanto en alcanzar la mayor exactitud posible como en comprender el proceso de clasificación y poder interpretar sus resultados.

El análisis exploratorio que se realizó previamente sobre estos mismos datos ya había dejado algunas señales útiles para esta etapa. Por un lado, la cantidad de artículos está fuertemente desbalanceada entre medios, con **Reuters** concentrando una porción muy superior al resto, algo que conviene tener presente tanto al entrenar los modelos como, sobre todo, al interpretar sus métricas. Por otro, se vio que los propios artículos suelen incluir pistas directas de su origen, como menciones al nombre del medio, direcciones web o firmas editoriales, que de no removerse podrían facilitar la tarea de forma artificial. Y aunque los medios comparten temáticas transversales, como la política estadounidense, mostraban perfiles de vocabulario diferenciados: **Reuters** aparecía ligado a un lenguaje económico e internacional, mientras que medios como **The New York Times** o **People** se inclinaban hacia asuntos más cercanos a la vida cotidiana. Estas diferencias sugieren, ya de entrada, que habrá medios más fáciles de distinguir que otros.

Para esta tarea primero se traducirá el texto a una representación numérica con la que un modelo pueda operar, explorando técnicas como *bag of words* y *TF-IDF*, y observando mediante una reducción de dimensionalidad hasta qué punto los medios resultan separables. Luego se entrena y evalúa el rendimiento de un modelo *Multinomial Naive Bayes*, evaluando su desempeño con métricas de clasificación.

Dataset y representación numérica de texto

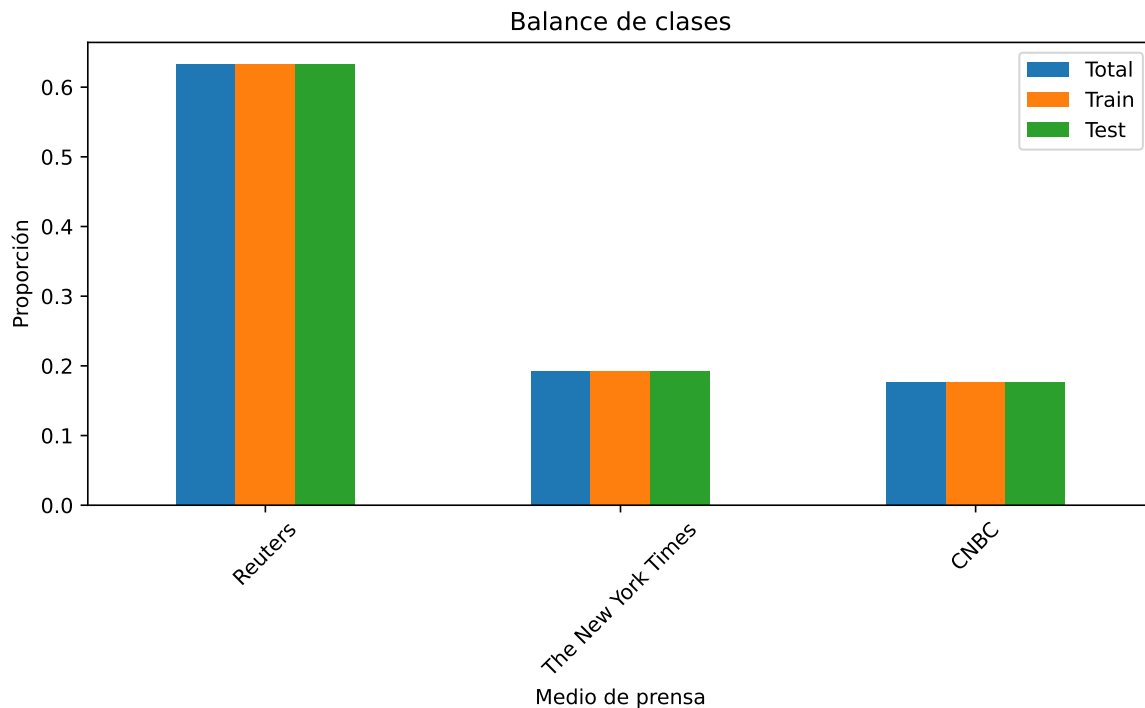
Preparación de los datos

Se utilizará un conjunto de datos reducido de los **tres medios de prensa con mayor cantidad de artículos**.

Particione los datos para generar un conjunto de test del 30 % del total, utilizando muestreo estratificado.

Sugerencia: utilice el parámetro `stratify` de la función `train_test_split` de scikit-learn y fije también el valor de `random_state` para obtener resultados reproducibles.

2. Verificación del balance de clases



3. Representación Bag of Words

Transforme el texto del conjunto de entrenamiento a una representación numérica (features) de conteo de palabras (*bag of words*). Explique brevemente cómo funciona esta técnica y muestre

un ejemplo. En particular, explique el tamaño de la matriz resultante y la razón por la que es una matriz *sparse*.

Sugerencia: puede ser útil imaginar qué sucedería con la memoria RAM requerida si no estuviéramos trabajando con un conjunto de datos reducido.

Respuesta

El funcionamiento de la técnica BOW se basa en generar una matriz donde cada fila representa, en este caso, un artículo y cada columna una palabra distinta que aparezca en el conjunto de todos los artículos del dataset. Los elementos de la matriz son el conteo de la respectiva palabra en el correspondiente artículo.

Como tenemos un set de entrenamiento y otro de testeo, tendremos 2 matrices. En ambos casos las columnas representan las distintas palabras que aparecen en los artículos del set de entrenamiento. La matriz generada para el set de entrenamiento es de dimensiones (10262, 82287) y la generada para el set de testeo de (4398, 82287).

Si no es necesario para la parte 2 procesar el set de testeo, borrar esto del texto.

Veamos como ejemplo los primeros 5 artículos de la matriz correspondiente al set de entrenamiento. Elegimos las palabras de forma que al menos un elemento sea distinto de 0:

Tabla 1: Ejemplo de representación Bag of Words.

los	angeles	reuters	star	wars	actress
4	4	1	4	3	3
0	0	1	0	0	0
0	0	1	0	0	0
0	0	0	0	0	0
0	0	0	0	0	0

Tomando en cuenta que la cantidad promedio de palabras por artículo en nuestro dataset, de únicamente los 3 medios de prensa con más publicaciones, es $424 \ll 82287$, es comprensible suponer que ambas matrices son esparzas, ya que la mayoría de elemento serán entonces 0. Es aún más fácil de ver si comparamos ahora con la cantidad de palabras en el artículo que cuenta con el mayor número de palabras, $12069 < 82287$, es decir que aún en la fila correspondiente a este artículo, la amplia mayoría de elementos serán 0.

Si almacenáramos estas matrices en la forma clásica, habría que guardar $10262 \times 82287 = 844.429.194$ valores y $4398 \times 82287 = 361.898.226$ valores, respectivamente, lo cual ocuparía mucha memoria, por lo que utilizaremos el hecho de que ambas son matrices esparzas a favor para almacenarlas de manera más eficiente: utilizaremos la representación *sparse* de *scikit-learn*, la cual almacena solo las posiciones y valores de los elementos distintos de cero.

4. Representación TF-IDF

Explique brevemente qué es un **n-grama**. Obtenga la representación numérica *Term Frequency - Inverse Document Frequency* (TF-IDF). Explique brevemente en qué consiste esta transformación adicional.

Respuesta

Un **n-grama** consiste en una secuencia de n elementos contiguos. En nuestro caso, se podría usar como alternativa a la representación BOW, tomando, en lugar de todas las palabras simples, todas las secuencias de largo n de palabras de los artículos, con n a determinar. A este tipo de representaciones se las conoce como **Bag-of-ngrams**. Se podría decir que la representación BOW es solo un caso particular de esta donde $n = 1$.

La representación TF-IDF ataca un problema presente en la representación BOW: los conteos absolutos se ven afectados por el largo del texto. En general se espera que en promedio textos más largos presenten más veces cada palabra. El componente TF de esta nueva representación se encarga de devolver la frecuencia relativa de cada palabra por texto, es decir que:

$$TF_{t,p} = \frac{\text{conteo de la palabra } p \text{ en el texto } t}{\text{conteo total de palabras en el texto } t}$$

Además se agrega un componente que pondera cada palabra de un cierto texto por el logaritmo de la razón entre el total de palabras en todos los textos y la cantidad de veces que aparece dicha palabra en todo el conjunto de textos:

$$IDF_p = \frac{\text{cantidad total de textos}}{\text{cantidad de textos que contienen la palabra } p}$$

Su efecto sobre las palabras genera que aquéllas que aparecen en muchos textos cuenten con un valor relativamente menor, entendiéndose que estas son más genéricas (por ejemplo: “the”, “and”, “a”), mientras que las que aparecen en un menor número de textos son más informativas sobre el contexto de los mismos y por ende cuentan con un valor relativamente mayor.

La representación *Term Frequency - Inverse Document Frequency* se calcula entonces como el producto entre los dos componentes anteriores:

$$TF \times IDF$$

Continuando con las mismas palabras del ejemplo anterior, vemos esta vez su representación TF-IDF:

Tabla 2: Ejemplo de representación TF-IDF.

los	angeles	reuters	star	wars	actress
0.1289	0.1304	0.0108	0.1272	0.1223	0.1248
0.0000	0.0000	0.0251	0.0000	0.0000	0.0000
0.0000	0.0000	0.0097	0.0000	0.0000	0.0000
0.0000	0.0000	0.0000	0.0000	0.0000	0.0000
0.0000	0.0000	0.0000	0.0000	0.0000	0.0000

La mayoría de las palabras de ejemplo, donde el valor de representación no es nulo, cuentan con valores en torno a 0.125, la única palabra que se diferencia en este caso es “Reuters”, nombre de uno de los medios de prensa que publica los artículos. Es posible que la misma cuente con un valor menor, ya que no es una palabra muy frecuente dentro de cada texto del ejemplo particular, por lo que su valor de *TF* sería bajo. Pero también puede deberse a que el medio de prensa es uno de los que más se menciona a sí mismos en los artículos que publica, sumado a que es el medio con el mayor número de artículos publicados (ambos resultados se vieron en la tarea 1 entregada anteriormente), por lo que su *IDF* la ponderaría con un valor relativamente más bajo.

5. Visualización PCA sobre TF-IDF

Muestre en un mapa el conjunto de entrenamiento, utilizando las dos primeras componentes PCA sobre los vectores de TF-IDF. Analice los resultados y compare qué sucede si utiliza:

- a) el filtrado de `stop_words` para idioma inglés;
- b) el parámetro `use_idf=True`;
- c) `ngram_range=(1,2)`.

Opcionalmente, también puede analizar qué sucede si no elimina los signos de puntuación.

¿Se pueden separar los medios de prensa utilizando sólo 2 componentes principales? Haga una visualización que permita entender cómo varía la varianza explicada a medida que se agregan componentes (por ejemplo, hasta 10 componentes).

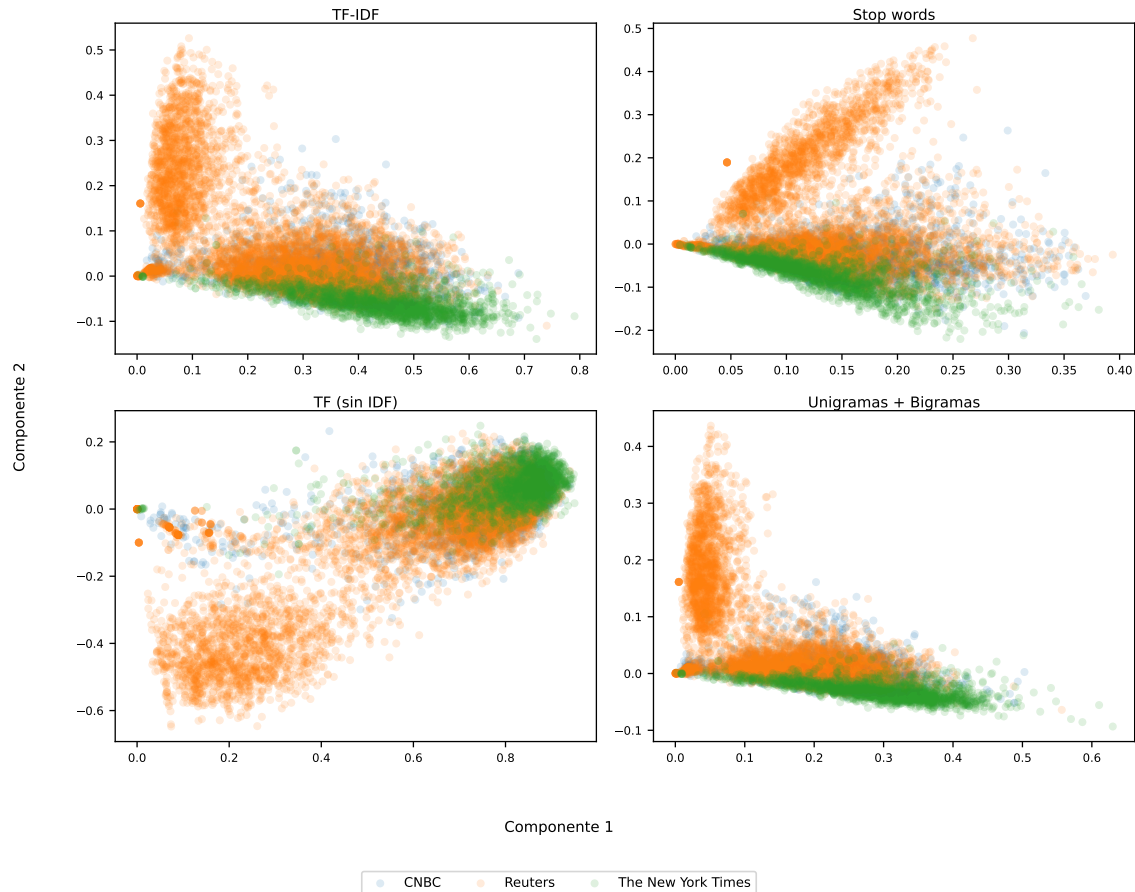
Discuta además si la separación observada puede deberse a diferencias de estilo editorial, a diferencias temáticas o a pistas explícitas del medio que no hayan sido removidas en la limpieza.

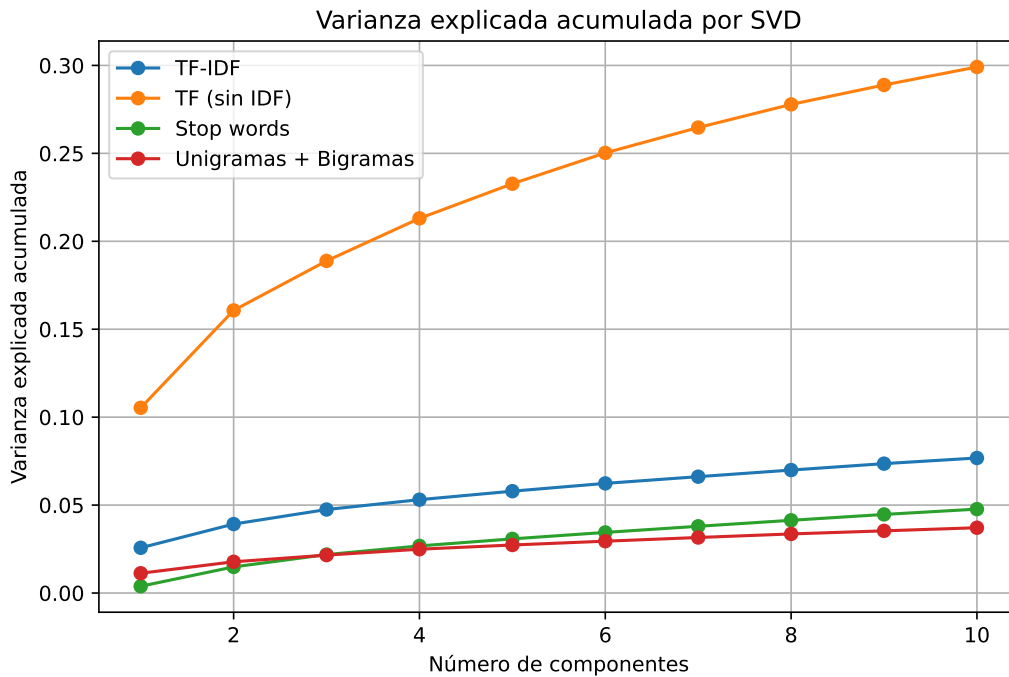
Respuesta

Observación 1: la función *TfidfVectorizer* usa por defecto `use_idf=True`, por lo que el ejemplo presentado en la tabla 2 cuenta con el componente *IDF* en su representación. A continuación

mostraremos los resultados, tanto con el argumento *use_idf=False*, donde solo se aplica un TF normalizado, como con *use_idf=True*.

Observación 2: Computacionalmente no es recomendable aplicar PCA, ya que la matriz resultante tiene dimensiones demasiado grandes y, para aplicar PCA, es necesario primero convertir la matriz esparza en una matriz densa. Por esto, en lugar de PCA, aplicaremos SVD. Esta segunda técnica no necesita de centrar la matriz, por lo que puede trabajar directamente con matrices esparzas.





Parte 2: Entrenamiento y Evaluación de Modelos

1. Multinomial Naive Bayes

Entrene el modelo *Multinomial Naive Bayes* para clasificar los artículos según a qué medio de prensa pertenece el texto. Utilice dicho modelo para clasificar los artículos del conjunto de test, y reporte el valor de *accuracy* y la **matriz de confusión**. Reporte además el valor de *precision* y *recall* para cada medio. Explique cómo se relacionan estos valores con la matriz anterior.

¿Qué problemas puede tener el hecho de mirar solamente el valor de *accuracy*? Considere qué sucedería con esta métrica si el desbalance de datos fuera aún mayor entre medios.

Sugerencia: utilice el método `from_predictions` de `ConfusionMatrixDisplay` para realizar la matriz.

3. Entrenamiento final con el mejor modelo

Elija el mejor modelo (mejores parámetros) y vuelva a entrenar sobre todo el conjunto de entrenamiento disponible (sin quitar datos para validación). Reporte el valor final de las

métricas y la matriz de confusión. Discuta las limitaciones de utilizar un modelo basado en *bag of words* o TF-IDF para el análisis de texto.

4. Modelo alternativo

Evalúe al menos un modelo más (dentro de scikit-learn) aparte de *Multinomial Naive Bayes* para clasificar el texto utilizando las mismas *features* de texto. Explique brevemente cómo funciona y compare los resultados con el anterior.

5. Cambio de medio de prensa

Evalúe el problema cambiando al menos un medio de prensa. En particular, observe el (des)balance de datos y los problemas que pueda generar, así como cualquier indicio que pueda ver en el mapeo previo con PCA. Puede ser útil comentar acerca de técnicas como sobre-muestreo y submuestreo; no es necesario implementarlas.

6. Técnica alternativa de extracción de features

Busque información sobre al menos una técnica alternativa de extraer *features* de texto. Explique brevemente cómo funciona y qué tipo de diferencias esperaría en los resultados. No se espera que implemente nada en esta parte.

TODO: Escriba su análisis en el informe.

7. Opcional: Clasificación a nivel de título

Repita la clasificación con los tres medios de prensa originales, pero esta vez clasificando a nivel de **título** en lugar de artículo completo.